

基于流网络的生成模型用于非迭代式多样化候选生成

Emmanuel Bengio^{1,2}, Moksh Jain^{1,5}, Maksym Korablyov¹
Doina Precup^{1,2,4}, Yoshua Bengio^{1,3}

¹Mila, ²McGill University, ³Université de Montréal, ⁴DeepMind, ⁵Microsoft

摘要

本文讨论的是从一系列动作中生成一个对象（如分子图）的学习随机策略问题，使得生成该对象的概率与其给定的正奖励成比例。而标准回报最大化往往收敛到单一的回报最大化序列，在某些情况下，我们希望采样一组多样化的高回报解决方案。这种情况出现在黑盒函数优化中，当可能的轮次较少，每一轮包含大量查询批次时，这些批次应该具有多样性，例如在新分子的设计中。这也可以被视为将能量函数近似转换为生成分布的问题。虽然 MCMC 方法可以实现这一点，但它们成本高昂，并且通常仅执行局部探索。相反，训练生成策略可以在训练过程中摊销搜索的成本，并实现快速生成。借鉴时间差分学习的洞见，我们提出了 GFlowNet，基于将生成过程视为流网络的观点，使其能够处理不同轨迹可以产生相同最终状态的情况，例如，有多种方式可以顺序添加原子来生成某个分子图。我们将轨迹集视为流，并将流一致性方程转化为学习目标，类似于将贝尔曼方程转化为时间差分方法。我们证明了所提出的任何全局最小目标都会生成一个从期望分布中采样的策略，并在存在许多模式的奖励函数的简单领域以及分子合成任务上展示了 GFlowNet 的改进性能和多样性。

1 引言

预期收益的最大化 R 在强化学习 (RL) 中通常通过将策略的所有概率质量放在收益最高的动作序列上实现 π 而在本文中，我们研究的目标不是生成单个最高奖励的动作序列，而是采样一个轨迹分布，其概率与给定的正回报或奖励函数成比例。这在探索很重要的任务中是有用的，即我们希望从回报函数的主要模式中进行采样。这相当于将能量函数转化为相应的生成模型的问题，其中要生成的对象是通过一系列动作获得的。通过改变能量函数的温度（即乘法缩放）或对回报取幂，可以控制生成器的选择性，即在低温下仅从最高模式周围生成，或者在较高温度下进行更多探索。这种设置的一个动机应用是迭代黑盒优化，其中学习者可以访问一个能够在每轮中计算大量候选者的奖励的预言机，例如，在药物发现应用中。当预言机本身不确定时，生成候选者的多样性尤为重要，例如，它可能由细胞测定组成，这是一种临床试验的廉价替代品。

，或者它可能由对接仿真的结果组成（估计候选小分子与目标蛋白质结合的程度），这是一种更准确但成本更高的下游评估（如细胞测定或小鼠体内测定）的替代品。

当调用预言机的成本高昂（例如涉及生物实验时），一种标准方法（Angermueller 等，2020）是在此类探索设置中应用机器学习的方式是利用已从预言机收集的数据（假设是一组 (x, y) 对，其中 x 是一个候选解决方案，而 y 是从预言机获得的对 x 的标量评估）来训练一个监督代理 f （视为模拟器），该代理预测 y 来自 x 。函数 f 或其变体 $\hat{f}$ ，如贝叶斯优化（Srinivas 等，2010；Negoescu 等，2011）中那样，可以用来作为奖励函数 R ，用于训练生成模型或策略，从而产生下一组实验测试的候选者。仅仅寻找最大化 $R(x)$ 的 x 是不够的，因为我们希望在批次查询中采样一组具有高值的 x ，这些值位于 $R(x)$ 模式周围的 R , i.e.,。请注意，存在其他获取多样性的方法，例如批量贝叶斯优化（Kirsch 等，2019）。所提方法的优势在于计算成本与批次大小呈线性关系（与比较候选对的方法不同，后者至少是二次的）。考虑到使用合成生物学可能进行十万次候选实验，线性扩展将是一个巨大的优势。

因此，在本文中，我们专注于特定的机器学习问题，即将给定的正奖励或回报函数转化为一个生成策略，该策略以概率比例于回报进行采样。

该概率与回报成正比。在上述应用中，我们仅在生成候选后才应用奖励函数，即除了终端状态外，奖励为零，回报为终端奖励。我们处于所谓的 RL（强化学习）阶段性设置中。所提出的方案将给定状态下分配给动作的概率视为一个网络中的流，该网络的节点是状态，从该节点出发的出边是由动作驱动的确定性转换（不要与归一化流混淆；Rezende 和 Mohamed (2016)）。流入网络的总流量是终端状态中的奖励之和（即，一个分区函数），并且可以证明它等于根节点（或起始状态）处的流量。所提出的算法受到贝尔曼更新的启发，并在每个状态的流入和流出流量匹配时收敛。选择动作的概率与其对应的出流量成比例的策略被证明能够实现期望的结果，即采样终端状态的概率与其奖励成比例。此外，我们展示了这种设置是离策略的；即使训练轨迹来自不同的策略，只要其在状态空间上有足够的支持，它仍然会收敛到上述解决方案。本文的主要贡献如下：

- 我们提出了 GFlowNet，一种基于流网络和局部流匹配条件的未归一化概率分布的新生成方法：流入状态的流量必须与流出流量相匹配。
- 我们证明了 GFlowNet 的关键属性，包括流匹配条件（许多训练目标可以提供）与生成策略与目标奖励函数匹配之间的联系。我们还证明了它的离线性质和渐近收敛性（如果可以最小化训练目标）。我们还展示了之前的相关工作（Buesing 等，2019），将生成过程视为一棵树，在存在许多可以导致相同状态的动作序列时会失败。
- 我们通过合成数据展示了偏离寻求回报的一种模式，转而寻求建模整个分布及其所有模式的有用性。
- 我们成功地将 GFlowNet 应用于大规模分子综合领域，并与 PPO 和 MCMC 方法进行了比较实验。

All implementations are available at <https://github.com/bengioe/gflownet>.

2 使用类似 TD 的目标近似流网络生成模型

考虑一个离散集合 $\mathcal{X}$ 和策略 $\pi(a|s)$ 来依次构建 $x \in \mathcal{X}$ ，其概率为 $\pi(x)$ 。

$$\pi(x) \approx \frac{R(x)}{Z} = \frac{R(x)}{\sum_{x' \in \mathcal{X}} R(x')} \quad (1)$$

其中， $R(x) > 0$ 是对终止状态 x 的奖励。这在给定一个基于分子化学性质评分的奖励函数 R 时，对于采样类似药物的新分子非常有用。能够从 $R(x)$ 的高模式中采样可以提供生成分子批次的多样性，这些分子将被发送到检测中。这与通常的强化学习目标——最大化回报不同，我们发现这通常会导致集中在一种或极少数几种好的分子上。在我们的上下文中， $R(x)$ 是实际从检测中获得值的代理，这意味着它可以频繁且廉价地调用。每当从检测中获取新数据时，都会重新训练或微调 $R(x)$ 。

应使用哪种方法来生成从 $\pi(x) \propto R(x)$ 采样的批次？让我们首先考虑一下我们将操作的状态空间。

令 $\mathcal{S}$ 表示状态集，令 $\mathcal{X} \subset \mathcal{S}$ 表示终止状态集。令 $\mathcal{A}$ 是一个有限集合，即字母表，令 $\mathcal{A}(s) \subseteq \mathcal{A}$ 是在状态 s 下允许的动作集，并令 $\mathcal{A}^*(s)$ 是状态 s 之后所有允许的动作序列集。对于动作序列集 $\{a_i \in \mathcal{A}, h \leq H\}$ 中的每一个动作序列 $\vec{a} = (a_1, a_2, a_3, \dots, a_h)$ ，都对应一个唯一的 x ，即环境是确定性的，我们可以定义一个函数 C 将动作序列 $\vec{a}$ 映射到一个 x 。如果这样的序列是‘不完整的’，则将其奖励定义为 0。当动作序列与状态之间存在一一对应关系时，状态 s 可以通过某个序列 $\vec{a}$ 唯一描述，我们可以将生成过程可视化为从单个根节点遍历到与动作序列对应的叶节点的过程。

然而，当这种对应关系是非单射的，即多个动作序列描述相同的 x 时，情况会变得复杂。我们得到的是一个有向无环图或 DAG（假设这些序列必须是有限长度的，即没有确定性的循环），如图 1 所示。例如，在这里感兴趣的分子可以被视为图，可以用多种顺序来描述（规范表示方法如 SMILES 字符串也存在这个问题：同一个实际分子可能有多种描述）。对于这样的采样问题，标准的方法是使用迭代 MCMC 方法（Xie 等，2021；Grathwohl 等，2021）。另一种选择是放松对拥有 $p(x) \propto R(x)$ 的需求，并使用非交互式（顺序）RL 方法（Gottipati 等，2020），但这些方法有很高的风险陷入局部最大值并错过模式。确实，在我们的设置中，最大化期望回报（即最终奖励的期望）的策略生成具有最高回报的序列（即单一分子）。

2.1 流网络

在本节中，我们提出了生成流网络框架，或 GFlowNet，使我们能够学习策略，使得 $p(x) \propto R(x)$ 在采样时成立。我们首先讨论现有方法为何不足，然后展示如何使用流、汇和源的比喻来构建适当的策略。接着，我们将展示可以通过流匹配目标来学习此类策略。

在双射情况下，使用现有方法，可以将一个 x 的顺序生成视为树结构确定性 MDP 中的一个情节，其中所有叶子 x 都是终端状态（奖励为 $R(x)$ ），而根节点是初始状态 s_0 。有趣的是，在这种情况下，可以将状态 $\tilde{V}(s)$ 的伪值表示为其所有后代的奖励之和 s （Buesing 等，2019）。

在非单射情况下，这些方法是不充分的。根据 Buesing 等（2019）、最大熵 RL（Haarnoja 等，2017）或通过自回归方法（Nash 和 Durkan，2019；Shi 等，2021）构造 $\pi(\tau) \approx R(\tau)/Z$ ，e.g. as 有一个特定的问题如下：如果多个动作序列 $\vec{a}$ （即多条轨迹 τ ）导致最终状态 x ，那么会在生成概率中引入严重的偏差。让我们记 $\vec{a} + \vec{b}$ 为两个动作序列 $\vec{a}$ 和 $\vec{b}$ 的连接，并由此扩展到 $s + \vec{b}$ 为从状态 s 应用动作 $\vec{b}$ 到达的状态。

命题 1. 设 $C : \mathcal{A}^* \mapsto \mathcal{S}$ 将每个允许的动作序列 $\vec{a} \in \mathcal{A}^*$ 关联到一个状态 $s = C(\vec{a}) \in \mathcal{S}$ 。设 $\tilde{V} : \mathcal{S} \mapsto \mathbf{R}^+$ 将每个状态 $s \in \mathcal{S}$ 关联到 $\tilde{V}(s) = \sum_{\vec{b} \in \mathcal{A}^*(s)} R(s + \vec{b}) > 0$ ，其中 $\mathcal{A}^*(s)$ 是从状态 s 出发的所有允许延续的集合，而 $s + \vec{b}$ 表示结果状态，即 $\tilde{V}(s)$ 是从状态 s 可达的所有状态的奖励之和。考虑一个从空字符串对应的状态 $s_0 = C(\emptyset)$ 开始，并在状态 $s \in \mathcal{S}$ 选择一个允许动作 $a \in \mathcal{A}(s)$ 的概率为 $\pi(a|s) = \frac{\tilde{V}(s+a)}{\tilde{V}(s)}$ 的策略 π 。

记 $\pi(\vec{a} = (a_1, \dots, a_N)) = \prod_{i=1}^N \pi(a_i | C(a_1, \dots, a_{i-1}))$ 和 $\pi(s)$ 与 $s \in \mathcal{S}$ 表示该策略访问状态 s 的概率。则有以下结果：(a) $\pi(s) = \sum_{\vec{a} : C(\vec{a})=s} \pi(\vec{a})$ 。

$$\pi(s) = \sum_{\vec{a} : C(\vec{a})=s} \pi(\vec{a}).$$

若 (b) If C 是双射, 则 $\tilde{V}(s) \pi(s) = \bar{V}(s_0)$, 对于终止状态作为特殊情况, $\frac{R(x)}{\sum_{x \in \mathcal{X}} R(x)}$.
 $\pi(x) = \frac{R(x)}{\sum_{x \in \mathcal{X}} R(x)}$ (c) If C 是非单射的, 存在 $n(x)$ 个不同的动作序列 $\vec{a}_i$ s.t. $C(\vec{a}_i) = x$, 则 $\sum_{x' \in \mathcal{X}} \frac{R(x')}{n(x')R(x')}$.

证明见附录 A.1。在组合空间中, 如分子结构, 其中 C 是非单射的 (构造分子图的方式有很多), 随着轨迹长度的增加, 这可能会变得指数级糟糕。这意味着较大的分子被采样的概率会比较小的分子高得多, 仅仅是因为通往它们的路径更多。在这种情况下, 伪值 $\tilde{V}$ 误解了 MDP 的结构为一棵树, 导致了错误的 $\pi(x)$ 。

另一种方法是将 MDP 视为流网络, 即利用 MDP 的 DAG 结构, 学习一个流 F , 而不是估计伪值 $\tilde{V}$ 为后代奖励之和, 如下所述。我们将流网络定义为具有单一源点, 即根节点 (或初始状态) s_0 , 其流入量为 Z , 以及每个叶节点 (或终止状态) x 作为汇点, 其流出量为 $R(x) > 0$ 。我们用 $T(s, a) = s'$ 表示状态-动作对 (s, a) 导向状态 s' 。需要注意的是, 因为 C 不是双射, 即有些节点有多条路径 (动作序列) 到达, 所以一个节点可以有多个父节点, 即 $|\{(s, a) \mid T(s, a) = s'\}| \geq 1$, 除了根节点, 它没有父节点。我们用 $F(s, a)$ 表示节点 s 和节点 $s' = T(s, a)$, $F(s)$ 之间的流量, 对于通过节点 s^1 的总流量。这种构建方式如图 1 所示。

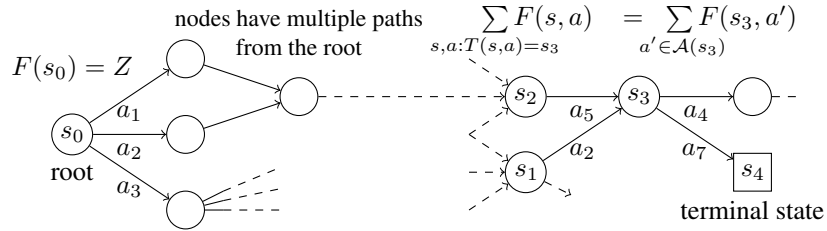


图 1: 一个流网络 MDP。回合从源点 s_0 开始, 带有流量 Z 。类似于 SMILES 字符串, 不存在循环。终止状态是具有流出量 $R(s)$ 的汇。示例样本状态 s_3 有父节点 $\{(s, a) \mid T(s, a) = s_3\} = \{(s_1, a_2), (s_2, a_5)\}$, 允许的动作 $\mathcal{A}(s_3) = \{a_4, a_7\}$ 。 s_4 是一个终端汇状态, 具有 $R(s_4) > 0$ 并且只有一个父节点。目标是估计 $F(s, a)$, 使得所有状态的流方程都得到满足: 对于每个节点, 流入流量等于流出流量。

为了满足流条件, 要求对于任意节点, 流入的流量等于流出的流量, 即节点 s 的总流量 $F(s)$ 。边界条件由终端节点 $x, R(x)$ 的流入流量给出。形式上, 对于任意节点 s' , 我们必须有流入流量

$$F(s') = \sum_{s,a:T(s,a)=s'} F(s, a) \quad (2)$$

equals the out-flow

$$F(s') = \sum_{a' \in \mathcal{A}(s')} F(s', a'). \quad (3)$$

更简洁地, 对于内部节点使用 $R(s) = 0$, 对于叶节点 (终端节点) 使用 $\mathcal{A}(s) = \emptyset$, 我们可以写出以下流量一致性方程:

$$\sum_{s,a:T(s,a)=s'} F(s, a) = R(s') + \sum_{a' \in \mathcal{A}(s')} F(s', a'). \quad (4)$$

其中 F 是一个流量, $F(s, a) > 0 \forall s, a$ (为此我们需要约束 $R(x)$ 也为正数)。原则上可以包含零流量的节点和边, 但这会使讨论流量的对数变得困难, 正如我们在下面所做的那样, 并且这些状态可以通过其父状态允许的动作集始终被排除。现在让我们证明, 在满足上述流量方程的情况下, 这样的流量能够正确生成 $\pi(x) = R(x)/Z$ 。

在某种意义上, $F(s)$ 和 $F(s, a)$ 接近于 V 和 Q , 即 RL 的价值函数和动作价值函数。这些函数有效地指导代理在 MDP 的每一步决策过程中以期望的方式行动。经过一些推导, 我们还可以证明 $F(s, a)$ 与某个策略 $\hat{\pi}$ 在修改后的 MDP 中的“真实” Q^π 之间的等价性 (见 A.2)。

命题 2. 定义策略
作 $a \in \mathcal{A}$ 根据

π 从状态开始生成轨迹

s_0 通过采样动

$$\pi(a|s) = \frac{F(s, a)}{F(s)} \quad (5)$$

其中 $F(s, a) > 0$ 是通过允许边 (s, a) , $F(s) = R(s) + \sum_{a \in \mathcal{A}(s)} F(s, a)$ 的流量
 $R(s) = 0$ 对于非终端节点 s 和 $F(x) = R(x) > 0$ 对于终端节点 x , 并且满足流量一致性方程
 $\sum_{s, a: T(s, a) = s'} F(s, a) = R(s') + \sum_{a' \in \mathcal{A}(s')} F(s', a')$ 。令 $\pi(s)$ 表示从状态 s 开始并遵循 $\pi(\cdot|\cdot)$ 访问状态 s 的概率。那么

$$\begin{aligned} (a) \pi(s) &= \frac{F(s)}{F(s_0)} \\ (b) F(s_0) &= \sum_{x \in \mathcal{X}} R(x) \\ (c) \pi(x) &= \frac{R(x)}{\sum_{x' \in \mathcal{X}} R(x')} \end{aligned}$$

证明。我们有 $\pi(s_0) = 1$, 因为我们总是从根节点开始 s_0 。需要注意的是 $\sum_{x \in \mathcal{X}} \pi(x) = 1$, 因为终端状态互斥, 但在非双射的情况下 C , 我们不能说 $\sum_{s \in S} \pi(s)$ 等于 1, 因为在一般情况下不同的状态并非互斥。这种表示方法与通常在强化学习中使用的表示方法不同, 在那里 $\pi(s)$ 指的是马尔可夫链的渐近分布。然后 $\pi(s') = \sum_{(a, s): T(s, a) = s'} \pi(a|s) \pi(s)$

$$\pi(s') = \sum_{(a, s): T(s, a) = s'} \pi(a|s) \pi(s) \quad (6)$$

即, 使用方程 5,

$$\pi(s') = \sum_{(a, s): T(s, a) = s'} \frac{F(s, a)}{F(s)} \pi(s). \quad (7)$$

我们现在猜想该陈述

$$\pi(s) = \frac{F(s)}{F(s_0)} \quad (8)$$

是正确的, 并通过归纳法进行证明。这对于我们基本的陈述——根节点来说显然是正确的, 因为 $\pi(s_0) = 1$ 。通过归纳法, 如果对于父节点 s 的 s' 该陈述成立, 则

$$\pi(s') = \sum_{s, a: T(s, a) = s'} \frac{F(s, a)}{F(s)} \frac{F(s)}{F(s_0)} = \frac{\sum_{s, a: T(s, a) = s'} F(s, a)}{F(s_0)} = \frac{F(s')}{F(s_0)} \quad (9)$$

这证明了该陈述, 即定理的第一个结论 (a)。然后我们可以将其应用于终端状态 x 的情况, 其流量固定为 $F(x) = R(x)$, 从而得到

$$\pi(x) = \frac{R(x)}{F(s_0)}. \quad (10)$$

注意到 $\sum_{x \in \mathcal{X}} \pi(x) = 1$ 并对等式的两边求和 10 超过 x 从而得到 (b),

$$F(s_0) = \sum_{x \in \mathcal{X}} R(x) \text{ 即。将其代入等式 10 从而得到 (c), 即 } \pi(x) = \frac{R(x)}{\sum_{x' \in \mathcal{X}} R(x')}. \quad \square$$

因此我们的选择为 $\pi(a|s)$ 满足我们的需求: 它将一个奖励函数映射 R 生成模型生成 x 以概率 $\pi(x) \propto R(x)$, 是否 C 是双射或非单射 (前者是后者的特殊情况, 我们已经给出了非单射情况的一般性证明)。

2.2 GFlowNet 的目标函数

我们现在可以利用强化学习的直觉, 基于上述理论结果创建一个学习算法。具体而言, 我们建议近似流 F , 使得在收敛时, 流一致性方程在我们的估计器 F 具有足够的容量下得到满足, 就像时间差分 (TD) 算法中的贝尔曼方程 (Sutton 和 Barto, 2018)。这可能会得出轨迹的如下目标函数 τ :

$$\tilde{\mathcal{L}}_{\theta}(\tau) = \sum_{s' \in \tau \neq s_0} \left(\sum_{s, a: T(s, a) = s'} F_{\theta}(s, a) - R(s') - \sum_{a' \in \mathcal{A}(s')} F_{\theta}(s', a') \right)^2. \quad (11)$$

从学习的角度来看，一个问题是在轨迹早期接近根节点的流动会非常大，而在轨迹晚期接近叶节点的流动则会非常小。在高维空间中，当 $\mathcal{X}$ 的基数呈指数级增长（例如，在形成一个 x ），所需的典型动作数量中），早期状态的 $F(s, a)$ 和 $F(s)$ 将比后期状态呈指数级更大。由于我们希望 $F(s, a)$ 成为神经网络的输出，这将导致严重的数值问题。

为避免此问题，我们在对数尺度上定义流匹配目标，在这里我们匹配的不是流入和流出的流，而是它们的对数，并训练我们的预测器估计 $(s, a) = \log F(s, a)$ ，并对 $F_\theta^{\log}$ 预测值进行指数求和对数运算来计算损失，从而得到对数差的平方：

$$\mathcal{L}_{\theta, \epsilon}(\tau) = \sum_{s' \in \tau \neq s_0} \left(\log \left[\epsilon + \sum_{s, a: T(s, a) = s'} \exp F_\theta^{\log}(s, a) \right] - \log \left[\epsilon + R(s') + \sum_{a' \in \mathcal{A}(s')} \exp F_\theta^{\log}(s', a') \right] \right)^2 \quad (12)$$

这使得梯度对大值和小值预测具有相同的权重。需要注意的是，匹配流量的对数等同于使流入与流出流量的比例更接近 1。为了给大流量的误差赋予更多权重并避免对极小数值取对数，我们将 $\log(\epsilon + (\text{流入流量}))$ 与 $\log(\epsilon + (\text{流出流量}))$ 进行比较。这不会改变全局最小值，即流量方程满足时的情况，但可以避免对极小流量取对数带来的数值问题。超参数 ϵ 也权衡了我们在匹配大流量与小流量之间施加的压力程度，在我们的实验中设置为接近其可取的最小值 R 。由于我们希望发现 R 的主要模式，因此更关注较大的流量是有意义的。还有许多其他目标函数，其中流量匹配也是全局最小值。

此类目标函数的一个有趣优势在于它们能够产生离线的非策略方法。预测的流量 F 不依赖于用于采样轨迹的策略（除了样本应充分覆盖轨迹空间以获得泛化能力之外）。如下所述的形式化说明，我们可以使用任何具有广泛支持的策略来采样训练轨迹，并仍然获得正确的流量和生成模型，即训练可以是非策略的。

命题 3. 设轨迹 τ 用于训练 F_θ 从一个探索性策略中采样 F_θ 其支撑与最优化策略相同。在公式 5 中定义以确保一致流 F_θ 。如果公式 4 得到满足，则该流是一致的。还假设 $\exists \theta : F_\theta = F$ ，即，我们选择了一组足够丰富的预测器。设 F_θ 为预期训练损失的最小值点。设 F_θ 具有这样的性质：当流匹配时，它能达到最低可能值。首先，可以证明该性质在公式 12 的损失函数中得到满足。然后

$$F_{\theta^*} = F^*, \text{ and } L_{\theta^*}(\tau) = 0 \quad \forall \tau \sim P(\theta), \quad (13)$$

即，预期损失的全局最优解提供了正确的流量。如果 $\pi_{\theta^*}(a|s) =$

then we also have

$$\pi_{\theta^*}(x) = \frac{R(x)}{Z}. \quad (14)$$

证明见附录 A.1。需要注意的是，从强化学习的角度来看，该方法类似于异步动态规划（Sutton 和 Barto, 2018, § 4.5），这是一种离线的非策略方法，在每个状态被无限次访问的情况下可以收敛。

3 相关工作

Buesing 等人（2019）提出了训练策略生成状态的目标，其概率与奖励成正比，但所提方法仅在动作序列与状态之间存在双射关系时才有意义。相比之下，GFlowNet 适用于更一般的情况，即许多路径可以通向同一状态。通过许多 MCMC 方法（Grathwohl 等, 2021; Dai 等, 2020），可以实现按给定未归一化的正函数比例进行采样的目标。然而，在模式混合具有挑战性的情况下（例如，在高维空间中，具有良好分离的模式占据总体积的一部分），收敛到目标分布可能极其缓慢。相比之下，GFlowNet 不是迭代的，而是通过一个足够探索性的训练过程来摊销从这些模式中采样的挑战。这个问题出现在分子生成中，并已在这一背景下使用多种生成模型（Shi 等, 2020; Jin 等, 2020; Luo 等, 2021）以及 MCMC 方法（Seff 等进行了研究）。

等人, 2019; 谢等人, 2021), 强化学习 (Segler 等人, 2017; 曹和 Kipf, 2018; Popova 等人, 2019; Gottipati 等人, 2020; Angermueller 等人, 2020) 以及进化方法 (Brown 等人, 2004; Jensen, 2019; Swersky 等人, 2020)。这些方法中的一些依赖于给定的一组“正例”(高回报)来训练生成模型, 从而未能充分利用“负例”和测量值的连续性(某些示例应比其他示例更频繁地生成)。另一些则依赖于强化学习的传统回报最大化目标, 这往往集中在一种或几种主导模式上, 正如我们在实验中发现的那样。除了分子之外, 之前的工作通过强化学习 (Bachman 和 Precup, 2015) 或基于能量的生成对抗网络 (GAN) (戴等人, 2017) 非贪婪地生成数据。我们在 (12) 中提出的优化目标可能会使读者联想到控制作为推理的 Soft Q-Learning (Haarnoja 等人, 2017) 的目标, 不同之处在于我们将状态的所有父节点包含在流入中, 而 Soft Q-Learning 仅使用轨迹中的父节点。根据命题 1, Soft Q-Learning 诱导出不同的策略, 即 $P(\tau) \propto R(\tau)$ 而不是 $P(x) \propto R(x)$ 。更一般地说, 我们仅考虑确定性的生成设置, 而强化学习是一种适用于随机环境的更通用框架。在网络流与深度学习交叉领域的文献较为稀少, 主要关注解决最大流问题 (Nazemi 和 Omid, 2012; 陈和张, 2020) 或现有流网络内的分类 (Rahul 等人, 2017; Pektaş 和 Acarman, 2019)。最后, 在蒙特卡洛树搜索 (MCTS) 中考虑到搜索空间是一个有向无环图 (DAG) 而非树的概念, 称为置换 (Childs 等人, 2008), 也与所提出的方法有一定的联系。

4 实验结果

我们首先在一个足够小的人工领域验证 GFlowNet 是否如预期般工作, 该领域可以精确计算配分函数, 并将其恢复模式的能力与标准 MCMC 和 RL 方法进行比较, 发现其采样分布更接近归一化奖励。我们发现 GFlowNet (A) 收敛到 $\pi(x) \propto R(x)$, (B) 比 MCMC 和 PPO 方法需要更少的样本就能达到一定的性能水平, (C) 更快地恢复所有模式, 无论是从实际时间还是访问和查询的状态数量来看, 都优于 MCMC 和 PPO。然后我们在一个大规模领域测试 GFlowNet, 该领域包括生成小型药物分子图, 奖励估计它们与目标蛋白质的结合亲和力 (参见附录 A.3)。我们发现 GFlowNet 比基线方法更快找到更高奖励和更多样化的分子。

4.1 一个 (超) 网格领域

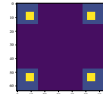
考虑一个马尔可夫决策过程 (MDP), 其中状态是维度为 n 的超立方体网格的单元格, 边长为 H 。智能体从坐标 $x = (0, 0, \dots)$ 开始, 并且仅允许通过动作 a_i (up to H 增加坐标 i (执行该动作后, 回合终止)。停止动作表示终止轨迹。有许多动作序列可以导致相同的坐标, 使得这个 MDP 是一个有向无环图 (DAG)。在坐标 x 结束轨迹的奖励是一些值 $R(x) > 0$ 。对于马尔可夫链蒙特卡洛 (MCMC) 方法, 为了使链具有遍历性, 我们允许迭代减少坐标, 并且没有停止动作。

我们使用以下奖励函数进行了实验:

$$R(x) = R_0 + R_1 \prod_i \mathbb{I}(0.25 < |x_i/H - 0.5|) + R_2 \prod_i \mathbb{I}(0.3 < |x_i/H - 0.5| < 0.4)$$

与 $0 < R_0 \ll R_1 < R_2$, 如所示当 $n = 2$ 在右侧。对于这种选择 R 在网格的角落附近只有有趣的奖励, 并且恰好有 2^n 模式。我们设定 $R_1 = 1/2$, $R_2 = 2$ 通过变化 R_0 将它设置得更接近于 0, 我们会人为地使这个问题变得更难, 从而创建一个状态空间中的区域, 探索这个区域是不理想的。为了衡量方法的性能, 我们测量经验 L1 误差 $\mathbb{E}[|p(x) - \pi(x)|]$, $p(x) = R(x)/Z$ 在该领域已知, π 通过重复采样并计算每种可能情况的频率来进行估计 x 我们也测度至少访问 1 次的模式数量作为访问状态数量的函数

我们在 $R_0 \in \{10^{-1}, 10^{-2}, 10^{-3}\}$ 条件下运行上述实验, 并使用 $n = 4, H = 8$ 。在图 2 中, 我们可以看到 GFlowNet 对 R_0 具有鲁棒性, 并获得了较低的 L1 误差, 而基于 Metropolis-Hastings-MCMC 的方法需要比 GFlowNet 多出指数级的样本才能达到相同的 L1 误差水平。这一点在图 2 (横坐标为对数尺度) 中通过比较 GFlowNet (初始阶段之后) 和 MCMC 采样的进度斜率可以明显看出。我们还观察到, 随着 R_0 减小, MCMC 访问每个模式所需的时间显著增加, 而 GFlowNet 仅受到轻微影响, 根据假设 (B), GFlowNet 更快地收敛到某个 L1 误差水平。这表明



GFlowNet 对模式之间的分离（由 R_0 变小表示）具有鲁棒性，因此比 MCMC 更快地恢复所有模式（再次注意到横坐标的对数尺度）。

为了与强化学习进行比较，我们运行了 PPO (Schulman 等人, 2017)。为了在合理时间内发现所有模式，我们需要将熵最大化项设置得远高于常规值（0.5）（ $\ll 1$ ）。我们通过将其与随机代理进行比较来验证 PPO 并未过度正则化。PPO 比均匀采样更快地找到所有模式，但比 GFlowNet 慢得多，并且对 R_0 的选择也具有鲁棒性。这一结果以及之前的结论验证了假设 (C)。我们还运行了 SAC (Haarnoja 等人, 2018)，得到了类似或更差的结果。我们在附录 A.6 中提供了额外的结果和讨论。

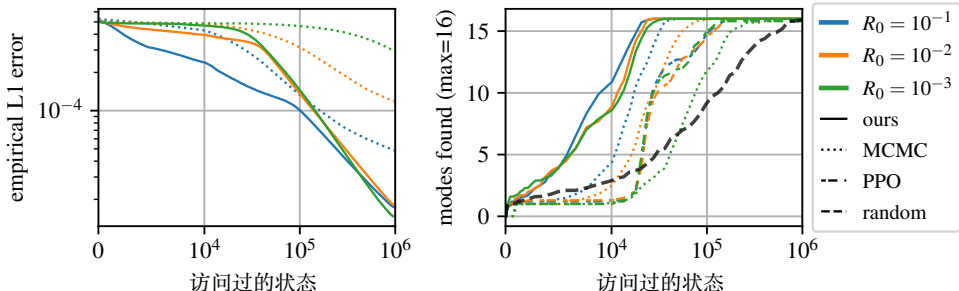


图 2: 超网格域。改变任务难度 R_0 以展示 GFlowNet 相对于其他方法的优势。我们可以看到，随着 R_0 变小，MCMC 难以拟合分布，因为它难以访问所有模式。PPO 也难以找到所有模式，并且需要非常大的熵正则化，但它对 R_0 的选择具有鲁棒性。我们绘制了每种设置下 10 次运行的平均值。

4.2 生成小分子 我们的目标是生成一组具有高奖励的小分子。我们定义了一个大规模环境，允许代理顺序生成分子。这个环境具有挑战性，状态数量高达 10^{16} ，动作数量在 100 到 2000 之间，具体取决于状态。我们遵循 Jin 等人 (2020) 的框架，通过预定义的构建块词汇表按部分生成分子，这些构建块可以连接形成一个交点树（详见附录 A.3）。这被称为基于片段的药物设计 (Kumar 等, 2012; Xie 等, 2021)。生成这样的图可以描述为一系列附加编辑：给定一个分子和化学有效性的约束条件，我们选择一个原子来连接一个构建块。因此，动作空间是选择连接位置和选择连接哪个构建块的乘积。还有一个额外的动作用于停止编辑序列。这一系列编辑形成了一个 DAG MDP，因为有许多不同的动作序列可以导致相同的分子图，并且没有移除边的动作，从而防止了循环。奖励由一个预训练的代理模型计算得出，该模型预测分子与特定蛋白质靶标（可溶性环氧化物水解酶，sEH，详见附录 A.3）的结合能量。尽管计算结合能量在计算上很昂贵，但我们可以通过调用这个代理模型来廉价地进行计算。需要注意的是，对于实际的药物设计，我们需要考虑许多其他因素，如类药性 (Bickerton 等, 2012)、毒性或合成性。我们的目标不是解决这个问题，而是将工作置于这样一个更大的项目中。相反，我们希望展示的是，在分子空间内给定一个代理 R ，我们可以快速匹配其诱导分布 $\pi(x) \propto R(x)$ 并找到其许多模式。我们使用 MPNN (Gilmer 等, 2017) 对原子图上的代理进行参数化。我们的流预测器 F_θ 类似于 MARS (Xie 等, 2021) 的参数化方式，但使用的是交点树图（构建块的图），这表现得更好。为了公平起见，这种架构被用于 GFlowNet 和基线方法。完整的细节可以在附录 A.4 中找到。

我们先用一个半整理半随机的数据集对代理进行预训练，该数据集包含 300k 个分子（参见 A.4），直到测试均方误差 (MSE) 降至 0.6；分子根据对接得分 (Trott 和 Olson, 2010) 进行评分，并重新归一化，使得大多数得分落在 0 到 10 之间（以拥有 $R(x) > 0$ ）。我们在图 3 中绘制了数据集的奖励分布。我们使用最多 10^6 个分子来训练所有生成模型。在训练过程中，采样遵循探索策略 $P(a|s)$ ，这是一种混合策略，其中以 0.95 的概率使用 $\pi(a|s)$ (Eq. 5)，以 0.05 的概率使用允许动作的均匀分布。

实验结果 在图 3 中，我们展示了两组设置下的奖励经验分布；首先是使用 $R(x)$ 训练模型的情况，然后是使用 $R(x)^\beta$ 的情况。如果 GFlowNet 学习了一个合理的策略 π ，

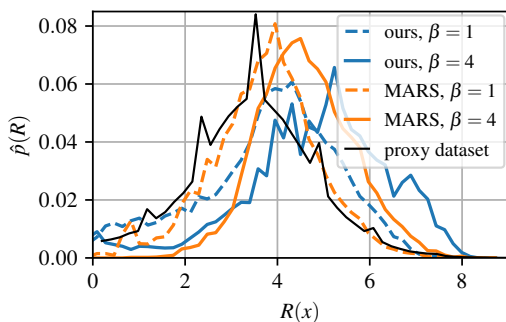


图 3: 奖励的经验密度。我们通过使用 R^β , $\beta = 4$ 来验证 GFlowNet 的一致性, 这确实将密度向右移动。

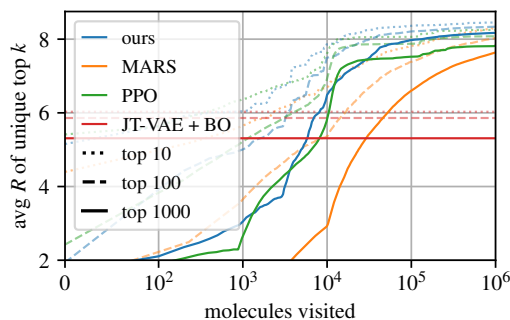


图 4: 学习过程中前 k 个分子的平均奖励 (基于 3 次运行的平均值)。仅计算唯一的命中次数。请注意对数尺度。我们的方法更快地找到更多的独特优质分子。

这应该将分布向右移动。我们确实观察到了这一点。我们将 GFlowNet 与已知在分子领域表现良好的 MARS (Xie 等人, 2021) 进行了比较, 并观察到了同样的移动。请注意, GFlowNet 找到的高奖励分子比 MARS 多, 这些 β 值一致表明它更快地找到高奖励模式 (由于 MARS 是一种 MCMC 方法, 最终会收敛到相同的分布, 但需要更多时间)。在图 4 中, 我们展示了迄今为止找到的前 k 个分子的平均奖励, 不允许重复 (基于 SMILES)。我们将 GFlowNet 与 MARS、PPO 和 JT-VAE (Jin 等人, 2020) 以及贝叶斯优化进行了比较。正如预期的那样, PPO 在一段时间后趋于平稳; 除非有强烈的探索机制进行正则化, 否则强化学习倾向于满足于足够好的轨迹。对于 GFlowNet 和 MARS, 访问的分子越多, 它们的表现越好, 逐渐趋近于代理的最大奖励。给定相同的时间, JT-VAE+BO 仅生成大约 10^3 个分子 (由于其昂贵的高斯过程), 因此表现不佳。

代理数据集中的最大奖励值为 10, 仅有 233 个样本超过 8。在我们最佳运行中, 训练过程中发现了 2339 个独特的分子, 其得分超过 8, 其中只有 39 个存在于数据集中。我们计算了得分最高的 1000 个样本的平均成对 Tanimoto 相似度: GFlowNet 的均值为 0.44 ± 0.01 , PPO 为 0.62 ± 0.03 , MARS 为 0.59 ± 0.02 (三次运行的均值和标准差)。正如预期, 我们的 MCMC 基线 (MARS) 和 RL 基线 (PPO) 找到的候选物多样性较低。我们还发现, GFlowNet 发现的模式数量远多于其他方法 (>1500 , 而 MARS 为 $R > 8$ vs <100)。这一点在图 5 中有所展示, 我们将模式定义为 Bemis-Murcko 支架 (Bemis 和 Murcko, 11996 年), 并统计了超过一定奖励阈值的分子数量。我们在附录 A.7 中提供了更多关于 GFlowNet 如何匹配奖励的洞见。

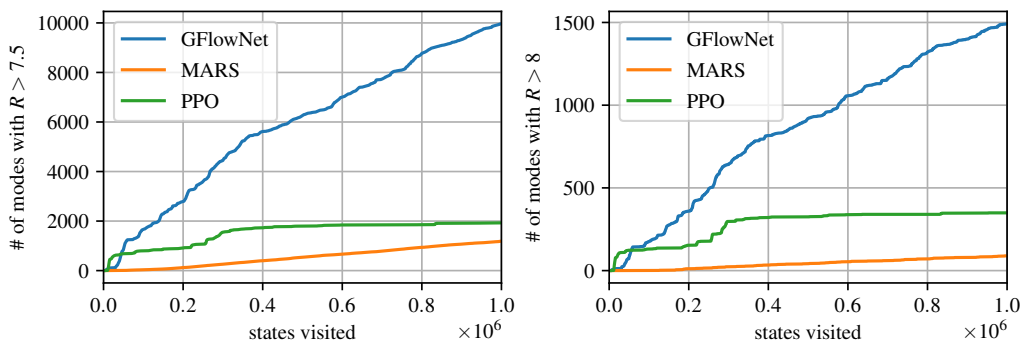


图 5: 超过奖励阈值的多样化 Bemis-Murcko 骨架数量 T 作为观察到的分子数量的函数。左图, $T = 7.5$ 。右图, $T = 8$ 。

4.3 多轮实验

为了展示在主动学习环境中生成多样候选的重要性, 我们考虑了一个顺序获取任务。我们模拟了一个场景, 在该场景中, 调用真实预言机的预算有限 O 。我们使用一个由有限数据集初始化的代理 M , 该数据集包含 $(x, R(x))$

对 D_0 ，其中 $R(x)$ 是来自预言机的真实奖励。生成模型 (π_θ) 被训练以拟合代理 M 学习到的未归一化概率函数。然后我们采样一批 $B = \{x_1, x_2, \dots, x_k\}$ ，其中 $x_i \sim \pi_\theta$ ，并使用预言机进行评估 O 。代理 M 通过新获得并标注的一批数据进行更新，这一过程重复进行 N 次迭代。我们在附录 A.5 中更详细地讨论了实验设置。

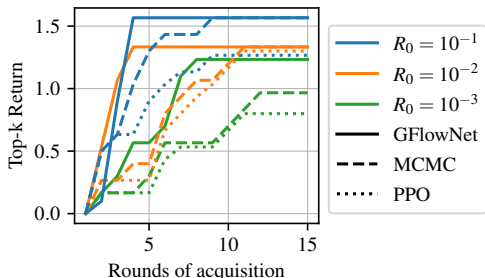


图 6: 在具有主动学习的 4 维超网格任务中，top-k 回报（三次运行的均值）。GFlowNet 更快地获得了最高回报。

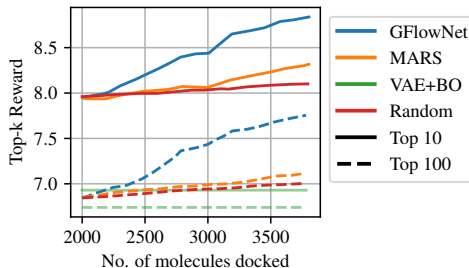


图 7: 在具有主动学习的分子任务中，top-k 对接奖励（三次运行的均值）。GFlowNet 始终生成更好的样本。

超网格领域 我们在图 6 中展示了 4 维超网格域中多轮任务的结果。我们使用高斯过程 (Williams 和 Rasmussen, 1995 年) 作为代理模型。我们将所有方法的 Top-k 回报进行比较，定义为 $\text{mean}(\text{top-}k(D_i)) - \text{mean}(\text{top-}k(D_{i-1}))$ ，其中 D_i 是从开始到第 i 步获得的数据点集， $k = 10$ 对于本次实验而言。初始数据集 D_0 ($|D_0| = 512$) 对所有比较的方法都是相同的。我们观察到 GFlowNet 在初始集合上的回报始终优于基线方法。我们还注意到，在最终轮次结束时，top- k 个点之间的平均成对 L2 距离是 0.83 ± 0.03 , 0.61 ± 0.01 并且 0.51 ± 0.02 对于 GFlowNet、MCMC 和 PPO 分别而言。这展示了 GFlowNet 即使在没有真实预言机的情况下也能捕捉模式的能力，以及在多轮设置中捕捉这种多样性的重要性。

小分子 对于分子发现任务，我们初始化一个 MPNN 代理来预测 AutoDock (Trott 和 Olson, 2010 年) 的对接分数，使用 $|D_0| = 2000$ 个分子。每轮结束时，我们生成 200 个分子，这些分子通过 AutoDock 进行评估，并用于更新代理。图 7 显示 GFlowNet 发现的分子能量显著高于初始集合 D_0 。它还始终优于 MARS 以及随机获取。由于 PPO 训练不稳定并持续发散，因此未报告其数值。初始集合中的平均成对 Tanimoto 相似度为 0.60。在最终轮次结束时，GFlowNet 和 MARS 的值分别为 0.54 ± 0.04 和 0.64 ± 0.03 。这进一步证明了 GFlowNet 生成多样化候选的能力，这最终有助于提高任务的最终性能。与单步设置类似，我们观察到 JT-VAE+BO 只能生成与计算时间相似的 10^3 个分子，因此表现不佳。

5 讨论与局限性

在这篇论文中，我们介绍了一种新颖的 TD 类目标，用于学习每个状态和（状态，动作）对的流，使得按这些流比例采样的策略可以按终端状态的奖励比例绘制终端状态。这可以被视为将能量函数转化为快速生成模型的一种替代方法，无需像 MCMC 方法那样需要迭代方法，并且具有当训练成功时，策略可以生成接近目标分布主要模式的大量样本的优势，而不会因模式间的混合问题而减慢速度。

局限性。 所提方法的一个缺点是，类似于基于 TD 的方法，使用引导可能导致优化挑战 (Kumar 等, 2020; Bengio 等, 2020)，并限制其性能。在药物发现等应用中，从每个模式周围的区域进行采样本身已经是一个重要优势，但未来的工作应探讨如何将这种生成方法与局部优化相结合，以精炼生成样本并接近奖励的局部最大值，同时保持候选批次的多样性。

负面社会影响。 作者未预见本工作的具体负面社会影响。

致谢和资金披露

本研究部分得益于 Calcul Québec (www.calculquebec.ca) 和 Compute Canada (www.computeCanada.ca) 提供的计算资源。所有作者的资金均来自其主要学术机构。作者还感谢三星电子株式会社、CIFAR 和 IBM 的资金支持。作者对 Andrei Nica 生成分子数据集、Maria Kadukova 在分子对接方面的建议、Harsh Satija 对论文的反馈表示感谢, 以及 Mila 分子发现团队的所有成员在我们面临的挑战上的许多研究讨论。

作者贡献

EB 和 YB 提出了原始想法, 并撰写了论文的大部分内容。YB 撰写了命题 1-3 的证明, EB 撰写了命题 4 的证明。EB 编写了第 4.1 节(超网格)和第 4.2 节(小分子)的实验代码并运行了实验。MJ 编写了第 4.3 节(多轮)的实验代码并运行了实验, 并撰写了论文中相应的结果部分。MK 构建了分子实验所基于的生化框架, 协助调试并运行了第 4.3 节的实验, 实现了用于第 4.2 节的模态计数程序, 并撰写了论文的生化细节。MK、DP 和 YB 为项目提供了指导。所有作者都参与了论文的校对和编辑。

参考文献

- Christof Angermueller, David Dohan, David Belanger, Ramya Deshpande, Kevin Murphy, and Lucy Colwell. Model-based reinforcement learning for biological sequence design. In *International Conference on Learning Representations*, 2020.
- Philip Bachman and Doina Precup. Data generation as sequential decision making. *Advances in Neural Information Processing Systems*, 28:3249–3257, 2015.
- Guy W Bemis and Mark A Murcko. The properties of known drugs. 1. molecular frameworks. *Journal of medicinal chemistry*, 39(15):2887–2893, 1996.
- Emmanuel Bengio, Joelle Pineau, and Doina Precup. Interference and generalization in temporal difference learning. In *International Conference on Machine Learning*, pages 767–777. PMLR, 2020.
- G Richard Bickerton, Gaia V Paolini, Jérémy Besnard, Sorel Muresan, and Andrew L Hopkins. Quantifying the chemical beauty of drugs. *Nature chemistry*, 4(2):90–98, 2012.
- Nathan Brown, Ben McKay, François Gilardoni, and Johann Gasteiger. A graph-based genetic algorithm and its application to the multiobjective evolution of median molecules. *Journal of chemical information and computer sciences*, 44(3):1079–1087, 2004.
- Lars Buesing, Nicolas Heess, and Theophane Weber. Approximate inference in discrete distributions with monte carlo tree search and value functions, 2019.
- Nicola De Cao and Thomas Kipf. Molgan: An implicit generative model for small molecular graphs, 2018.
- Yize Chen and Baosen Zhang. Learning to solve network flow problems via neural decoding. *arXiv preprint arXiv:2002.04091*, 2020.
- Benjamin E Childs, James H Brodeur, and Levente Kocsis. Transpositions and move groups in monte carlo tree search. In *2008 IEEE Symposium On Computational Intelligence and Games*, pages 389–395. IEEE, 2008.
- Hanjun Dai, Rishabh Singh, Bo Dai, Charles Sutton, and Dale Schuurmans. Learning discrete energy-based models via auxiliary-variable local exploration. In *Neural Information Processing Systems (NeurIPS)*, 2020.

- Zihang Dai, Amjad Almahairi, Philip Bachman, Eduard Hovy, and Aaron Courville. Calibrating energy-based generative adversarial networks. *arXiv preprint arXiv:1702.01691*, 2017.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry, 2017.
- Sai Krishna Gottipati, Boris Sattarov, Sufeng Niu, Yashaswi Pathak, Haoran Wei, Shengchao Liu, Karam M. J. Thomas, Simon Blackburn, Connor W. Coley, Jian Tang, Sarath Chandar, and Yoshua Bengio. Learning to navigate the synthetically accessible chemical space using reinforcement learning, 2020.
- Will Grathwohl, Kevin Swersky, Milad Hashemi, David Duvenaud, and Chris J. Maddison. Oops i took a gradient: Scalable sampling for discrete distributions, 2021.
- Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *International Conference on Machine Learning*, pages 1352–1361. PMLR, 2017.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018.
- Jan H Jensen. A graph-based genetic algorithm and generative model/monte carlo tree search for the exploration of chemical space. *Chemical science*, 10(12):3567–3572, 2019.
- Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Chapter 11. junction tree variational autoencoder for molecular graph generation. *Drug Discovery*, page 228–249, 2020. ISSN 2041-3211. doi: 10.1039/9781788016841-00228. URL <http://dx.doi.org/10.1039/9781788016841-00228>.
- Andreas Kirsch, Joost van Amersfoort, and Yarin Gal. Batchbald: Efficient and diverse batch acquisition for deep bayesian active learning, 2019.
- Ashutosh Kumar, A Voet, and KYJ Zhang. Fragment based drug design: from experimental to computational approaches. *Current medicinal chemistry*, 19(30):5128–5147, 2012.
- Aviral Kumar, Rishabh Agarwal, Dibya Ghosh, and Sergey Levine. Implicit under-parameterization inhibits data-efficient deep reinforcement learning, 2020.
- Greg Landrum. Rdkit: Open-source cheminformatics. URL <http://www.rdkit.org>.
- Youzhi Luo, Keqiang Yan, and Shuiwang Ji. Graphdf: A discrete flow model for molecular graph generation, 2021.
- Charlie Nash and Conor Durkan. Autoregressive energy machines, 2019.
- Alireza Nazemi and Farahnaz Omid. A capable neural network model for solving the maximum flow problem. *Journal of Computational and Applied Mathematics*, 236(14):3498–3513, 2012.
- Diana M. Negoescu, Peter I. Frazier, and Warren B. Powell. The knowledge-gradient algorithm for sequencing experiments in drug discovery. 23(3):346–363, 2011. ISSN 1526-5528. doi: 10.1287/ijoc.1100.0417. URL <https://doi.org/10.1287/ijoc.1100.0417>.
- Andrew Y Ng, Stuart J Russell, et al. Algorithms for inverse reinforcement learning. In *Icml*, volume 1, page 2, 2000.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. URL <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.

- Abdurrahman Pektaş and Tankut Acarman. Deep learning to detect botnet via network flow summaries. *Neural Computing and Applications*, 31(11):8021–8033, 2019.
- Mariya Popova, Mykhailo Shvets, Junier Oliva, and Olexandr Isayev. Molecularrnn: Generating realistic molecular graphs with optimized properties, 2019.
- RK Rahul, T Anjali, Vijay Krishna Menon, and KP Soman. Deep learning for network flow analysis and malware classification. In *International Symposium on Security in Computing and Communication*, pages 226–235. Springer, 2017.
- Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows, 2016.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- Ari Seff, Wenda Zhou, Farhan Damani, Abigail Doyle, and Ryan P Adams. Discrete object generation with reversible inductive construction. *arXiv preprint arXiv:1907.08268*, 2019.
- Marwin H. S. Segler, Thierry Kogej, Christian Tyrchan, and Mark P. Waller. Generating focussed molecule libraries for drug discovery with recurrent neural networks, 2017.
- Chence Shi, Minkai Xu, Zhaocheng Zhu, Weinan Zhang, Ming Zhang, and Jian Tang. Graphaf: a flow-based autoregressive model for molecular graph generation, 2020.
- Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjin Wang, and Yu Sun. Masked label prediction: Unified message passing model for semi-supervised classification, 2021.
- Niranjan Srinivas, Andreas Krause, S. Kakade, and M. Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. In *ICML*, 2010.
- Teague Sterling and John J Irwin. Zinc 15–ligand discovery for everyone. *Journal of chemical information and modeling*, 55(11):2324–2337, 2015.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- Kevin Swersky, Yulia Rubanova, David Dohan, and Kevin Murphy. Amortized bayesian optimization over discrete spaces. In *Conference on Uncertainty in Artificial Intelligence*, pages 769–778. PMLR, 2020.
- Oleg Trott and Arthur J Olson. Autodock vina: improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of computational chemistry*, 31(2):455–461, 2010.
- C. K. Williams and C. Rasmussen. Gaussian processes for regression. In *Neural Information Processing Systems (NeurIPS)*, 1995.
- Yutong Xie, Chence Shi, Hao Zhou, Yuwei Yang, Weinan Zhang, Yong Yu, and Lei Li. {MARS}: Markov molecular sampling for multi-objective drug discovery. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=kHSu4ebxFXY>.

A 附录

本文中所有机器学习代码均使用 PyTorch (Paszke 等, 2019) 库。我们重新实现了强化学习和其他基线方法。对于结合能估计, 我们使用 AutoDock Vina (Trott 和 Olson, 2010) 库, 并使用 RDKit (Landrum) 进行化学操作。运行本文中提出的全部分子实验预计需要 26 个 GPU 天。我们使用配备 NVidia V100 GPU 的集群。网格实验预计需要 8 个 CPU 天 (单核)。所有实现均可在 <https://github.com/bengiote/gflownet> 获取。

A.1 证明

命题 1. 设 $C : \mathcal{A}^* \mapsto \mathcal{S}$ 将每个允许的动作序列 $\vec{a} \in \mathcal{A}^*$ 关联到一个状态 $s = C(\vec{a}) \in \mathcal{S}$ 。设 $\tilde{V} : \mathcal{S} \mapsto \mathbf{R}^+$ 将每个状态 $s \in \mathcal{S}$ 关联到 $\tilde{V}(s) = \sum_{\vec{b} \in \mathcal{A}^*(s)} R(s + \vec{b}) > 0$, 其中 $\mathcal{A}^*(s)$ 是从状态 s 出发的所有允许延续的集合, 而 $s + \vec{b}$ 表示结果状态, 即 $\tilde{V}(s)$ 是从状态 s 可达的所有状态的奖励之和。考虑一个从空字符串对应的状态 $s_0 = C(\emptyset)$ 开始, 并在状态 $s \in \mathcal{S}$ 选择一个允许动作 $a \in \mathcal{A}(s)$ 的概率为 $\pi(a|s) = \frac{\tilde{V}(s+a)}{\tilde{V}(s)}$ 的策略 π 。

记 $\pi(\vec{a} = (a_1, \dots, a_N)) = \prod_{i=1}^N \pi(a_i | C(a_1, \dots, a_{i-1}))$ 和 $\pi(s)$ 与 $s \in \mathcal{S}$ 表示该策略访问状态 s 的概率。则有以下结果: (a) $\pi(s) = \sum_{\vec{a}_i: C(\vec{a}_i)=s} \pi(\vec{a}_i)$.

(b) If C is bijective, then $\pi(s) = \frac{\tilde{V}(s)}{\sum_{x \in \mathcal{X}} R(x)}$ and as a special case for terminal states x , $\pi(x) = \frac{R(x)}{\sum_{x \in \mathcal{X}} R(x)}$.

如果 (c) If C 是非单射的, 并且存在 $n(x)$ 不同的动作序列 $\vec{a}_i$ s.t. $C(\vec{a}_i) = x$, 那么 $\sum_{x' \in \mathcal{X}} \frac{R(x')}{n(x')R(x')}$ 。

证明。由于 s 可以从 s_0 通过任意的动作序列 $\vec{a}_i$ 到达, 这些序列满足 $C(\vec{a}_i) = s$ 并且它们互斥且覆盖了所有可能的到达 s 的方式, 因此 π 访问状态 s 的概率就是 $\sum_{\vec{a}_i: C(\vec{a}_i)=s} \pi(\vec{a}_i)$, 即我们得到 (a)。如果 C 是一一对应的, 这意味着只有一个这样的动作序列 $\vec{a} = (a_1, \dots, a_N)$ 能够到达状态 s , 并且动作序列和状态的集合形成了一个以 s_0 为根的树。因此根据 (a), 我们得到 $\pi(s) = \pi(\vec{a})$ 。首先注意到因为 $\tilde{V}(s) = \sum_{\vec{b} \in \mathcal{A}^*(s)} R(s + \vec{b})$, i.e., $\tilde{V}(s)$ 是

所有从 s 出发的动作序列的奖励之和, 我们有 $\tilde{V}(s) = \sum_{\vec{b} \in \mathcal{A}^*(s)} \tilde{V}(s + \vec{b})$ 当 $s = s_0$ 时成立, 因为 $\pi(s_0 = 1)$ (即每条轨迹都包含 s_0)。假设对于

$s' = C(a_1, \dots, a_{N-1})$ 成立, 考虑 $s = C(a_1, \dots, a_N)$:

$$\pi(s) = \pi(a_N | s') \pi(s') = \frac{\tilde{V}(s)}{\sum_{\vec{b} \in \mathcal{A}^*(s')} \tilde{V}(s' + \vec{b})} \frac{\tilde{V}(s')}{\tilde{V}(s_0)}.$$

利用上述结果 $\tilde{V}(s) = \sum_{\vec{b} \in \mathcal{A}^*(s)} \tilde{V}(s + \vec{b})$, 我们从而得到 $\tilde{V}(s')$ 与 $\sum$ 的抵消效果, 并获得

$$\sum_{\vec{b} \in \mathcal{A}^*(s')} \tilde{V}(s' + \vec{b})$$

$$\pi(s) = \frac{\tilde{V}(s)}{\tilde{V}(s_0)}, \quad (15)$$

证明递归成立。根据递归的定义我们知道 $\tilde{V}(s_0) = \sum_{x \in \mathcal{X}} R(x)$, 因此对于这个特殊情况 x 终态, $\tilde{V}(x) = R(x)$ 并见公式 15 变为 $\pi(x) = \frac{R(x)}{\sum_{x' \in \mathcal{X}} R(x')}$, 从而完成 (b) 的证明。另一方面, 如果 C 是非单射的, 路径集合形成一个有向无环图 (DAG), 而不是树。我们将通过创建一个新的状态空间 (用于树版本) 来将 DAG 转换成树, 这个状态空间就是动作序列本身。请注意, 同一个原始叶节点 x 现在重复出现 $n(x)$ 次于树中 (叶子由动作序列表示 $\vec{a}_i$), 如果有 $n(x)$ 动作序列导向 x 在有向无环图中。使用相同的定义 $\tilde{V}$ 和 $\pi(a|s)$ 但在树中, 我们得到 (b) 中的所有结果 (这些结果适用

由于我们有一个树结构), 特别是 $\pi(\vec{a}_i)$ 树下的比例与 $R(x') = R(x)$ 成正比。应用 (a), 我们可以看到 $\pi(x) \propto n(x)R(x)$, 这证明了 (c)。□

命题 3. 设轨迹 F_{θ^*} 用于训练, 从一个探索性策略中采样, 其支撑与最优策略相同。在公式 5 中定义以确保一致流。如果公式 4 得到满足, 则该流是一致的。还假设 $\exists \theta: F_{\theta} = F^*$, 即, 我们选择了一组足够丰富的预测器。设 F_{θ^*} 为预期训练损失的最小值点。设 $L_{\theta^*}(\tau) = 0 \quad \forall \tau \sim P(\theta)$ 。具有这样的性质: 当流匹配时, 它能达到最低可能值。首先, 可以证明该性质在公式 12 的损失函数中得到满足。然后

$$F_{\theta^*} = F^*, \quad \text{and} \quad (16)$$

$$L_{\theta^*}(\tau) = 0 \quad \forall \tau \sim P(\theta), \quad (17)$$

即, 预期损失的全局最优解提供了正确的流量。如果

$$\pi_{\theta^*}(a|s) = \frac{F_{\theta^*}(s, a)}{\sum_{a' \in \mathcal{A}(s)} F_{\theta^*}(s, a')} \quad (18)$$

于是我们也得到

$$\pi_{\theta^*}(x) = \frac{R(x)}{Z}. \quad (19)$$

证明。通过选择一个 θ 使得 $F_{\theta} = F^*$ (我们假设这是可能的), 可以实现每条轨迹损失为 0, 因为这使得流入流量等于流出流量。请注意, 在终端节点 (叶子节点) 仅对流量进行约束的情况下, 始终存在一个解 F^* 在允许的可能流量函数空间中满足流量方程 (流入 = 流出)。由于将 $L_{\theta}(\tau)$ 对于所有 $\tau \sim P(\theta)$ 设置为 0 可以使预期损失为 0, 并且这是可达到的最低值 (因为 $L_{\theta}(\tau) \geq 0 \quad \forall \theta$), 这意味着这样的 θ 是预期损失的全局最小值, 我们可以将其表示为 θ^* 。如果我们优化函数空间中的 F , 可以直接将损失关于 $F(s, a)$ 的梯度分别设置为 0, 并找到一个解。

由于我们选择了 F , 其支持范围足够大, 能够包含所有导向终止状态的轨迹 $R(x) > 0$, 这意味着对于这些轨迹上的所有节点, $L_{\theta}(\tau) = 0$ 成立, 并且 $F_{\theta} = F^*$ 也成立。然后我们可以应用命题 2 (因为流处处匹配, 并且我们根据公式 5 定义了策略)。通过应用命题 2 的结果 (c), 我们得到了结论。□

需要注意, 在一般情况下, 存在无限多的解。考虑两种可能的轨迹, 比如说 s_0, a_1, s_A, a_2, s_T 和 s_0, a_3, s_B, a_4, s_T , 它们都导向相同的终端状态 s_T , 并获得相同的奖励 r 。然后一个有效的解满足约束方程组 $F(s_A) + F(s_B) = r, F(s_A) > 0, F(s_B) > 0$, 我们可以看到存在由一个参数描述的无穷多解 u , 其中 $F(s_A) = u, F(s_B) = r - u, u \in [0, r]$ 。

A.2 动作值函数等价性

这里我们展示所提出的方法学习到的流 $F(s, a)$ 可以对应于强化学习意义上的“真实”的动作值函数 $Q^{\mu}(s, a)$, 对于某个策略 μ 而言。

首先注意到这一点在某种意义上是显然成立的: 在逆强化学习 (Ng 等人, 2000) 中, 通常存在无限多的解来定义从策略 π 得到的 $\hat{R}$, 使得 $\pi = \arg \max_{\pi_i} V^{\pi_i}(s; \hat{R}) \quad \forall s$, 其中 $V^{\pi_i}(s; \hat{R})$ 是在奖励函数 $\mathbf{R}$ 下状态 s 的价值函数。

更有趣的情况是, 从计算与 R 对应的流得到的 $F(s, a; R)$ 恰好等于某个 $Q^{\mu}(s, a; \hat{R})$ 乘以一个乘法因子 $f(s)$ 。那么 μ 和 $\hat{R}$ 是什么? 在双射的情况下有一个简单的答案。

命题 4. 设 μ 为均匀策略, 使得 $\mu(a|s) = 1/|\mathcal{A}(s)|$, 设 $f(x) = \prod_{t=0}^n |\mathcal{A}(s_t)|$ 当 $x \equiv (s_0, s_1, \dots, s_n)$, 并且设 $\hat{R}(x) = R(x)f(s_{n-1})$, 则 $Q^{\mu}(s, a; \hat{R}) = F(s, a; R)f(s)$ 。

证明. 根据动作值函数在下一步动作值上的定义以及
定义 μ :

$$Q^\mu(s, a; \hat{R}) = \hat{R}(s') + \frac{1}{|\mathcal{A}(s')|} \sum_{a' \in \mathcal{A}(s')} Q^\mu(s', a'; \hat{R}) \quad (20)$$

其中 $s' = T(s, a)$, 因为环境是确定性的, 并具有树结构。

对于某些叶节点 s' , $Q^\mu(s, a; \hat{R}) = \hat{R}(s') = R(s')f(s)$. 再次对于某些叶节点 s' , $F(s, a; R) = R(s')$, 流量为 $f(s)$. 因此 $Q^\mu(s, a; \hat{R}) = F(s, a; R)f(s)$. 假设 (20) 成立, 则通过归纳对于非叶节点 s :

$$Q^\mu(s, a; \hat{R}) = \hat{R}(s') + \frac{1}{|\mathcal{A}(s')|} \sum_{a' \in \mathcal{A}(s')} Q^\mu(s', a'; \hat{R}) \quad (21)$$

$$Q^\mu(s, a; \hat{R}) = 0 + \frac{1}{|\mathcal{A}(s')|} \sum_{a' \in \mathcal{A}(s')} F(s', a'; R)f(s') \quad (22)$$

we know from Eq 4 that

$$F(s, a; R) = \sum_{a' \in \mathcal{A}(s')} F(s', a'; R) \quad (23)$$

并且由于 $f(s') = f(s)|\mathcal{A}(s')|$, 我们得到:

$$Q^\mu(s, a; \hat{R}) = \frac{F(s, a; R)f(s')}{|\mathcal{A}(s')|} \quad (24)$$

$$= \frac{F(s, a; R)f(s)|\mathcal{A}(s')|}{|\mathcal{A}(s')|} \quad (25)$$

$$= F(s, a; R)f(s) \quad (26)$$

□

因此我们证明了双射情况下流量与均匀策略的动作值相对应。这一结果表明, 均匀策略的评估在树 MDP 情况下学到了一些非平凡的内容。或许这样的量可以以其他有趣的方式使用。在非单射的情况下, 由于存在无限数量的有效流量, 不清楚这种简单的等价关系总是存在。作为一个特定情况, 考虑流量 F , 它将恰好分配 0 流量给会导致任何节点有多条路径的边。换句话说, 考虑诱导出一棵树的流量, 即通过禁止不在该双射中的边之间的流量来实现动作序列和状态之间的双射。根据命题 4, 我们可以恢复某些有效 Q^μ . 由于至少存在一个这种等价关系的流量, 我们猜想更一般的流量和动作值函数之间的映射存在。

猜想 存在 f 一个关于 $n(s)$ 到达 s , $\mathcal{A}(s)$ 的路径数和 $n_p(s) = |\{(p, a) | T(p, a) = s\}|$ 节点 s 的父节点数的函数, 使得 $f(s, n(s), n_p(s), \mathcal{A}(s))Q^\mu(s, a; \hat{R}) = F(s, a; R)$ 并且 $\hat{R}(x) = R(x)f(x)$ 对于均匀策略 μ 和某个有效流量 $F(s, a; R)$ 。

A.3 分子领域细节

代理可以从一个包含 72 个预定义块的库中进行选择。从代理的角度复制这些块, 允许它们连接到给定块的不同对称群组上。这总共产生了每根茎 105 个动作; 茎是新块可以连接的原子。我们通过 Jin 等人 (2020) 提出的流程在 ZINC 数据集 (Sterling 和 Irwin, 2015) 上选择这些块。我们允许代理生成最多 8 个块。72 个块的 SMILES 包括 Br, C, C#N, C1=CCCCC1, C1=CNC=CC1, C1CC1, C1CCCC1, C1CCCCC1, C1CCNCC1, C1CCNCCC1, C1CCOC1, C1CCOCC1, C1CNCCN1, C1COCCN1, C1COCC[NH2+],

C=C, C=C(C)C, C=CC, C=N, C=O, CC, CC(C)C, CC(C)O, CC(N)=O, CC=O, CCC, CCO, CN, CNC, CNC(C)=O, CNC=O, CO, C[NH3+], C[SH2+], Cl, F, FC(F)F, I, N, N=CN, NC=O, N[SH](=O)=O, O, O=CNO, O=CO, O=C[O-], O=PO, O=P[O-], O=S=O, O=[NH+][O-],

O=[PH](O)O, O=[PH]([O-])O, O=[SH](=O)O, O=[SH](=O)[O-], O=c1[nH]cnc2[nH]cnc12,
O=c1[nH]cnc2c1NCCN2, O=c1cc[nH]c(=O)[nH]1, O=c1nc2[nH]c3cccc3nc-2c(=O)[nH]1,
O=c1nccc[nH]1, S, c1cc[nH+]cc1, c1cc[nH]c1, c1ccc2[nH]ccc2c1, c1ccc2cccc2c1, c1cccc1,
c1ccncc1, c1ccsc1, c1cn[nH]c1, c1cncnc1, c1cscn1, c1ncc2nc[nH]c2n1.

我们在图 8 中展示了这些构建块及其连接点。

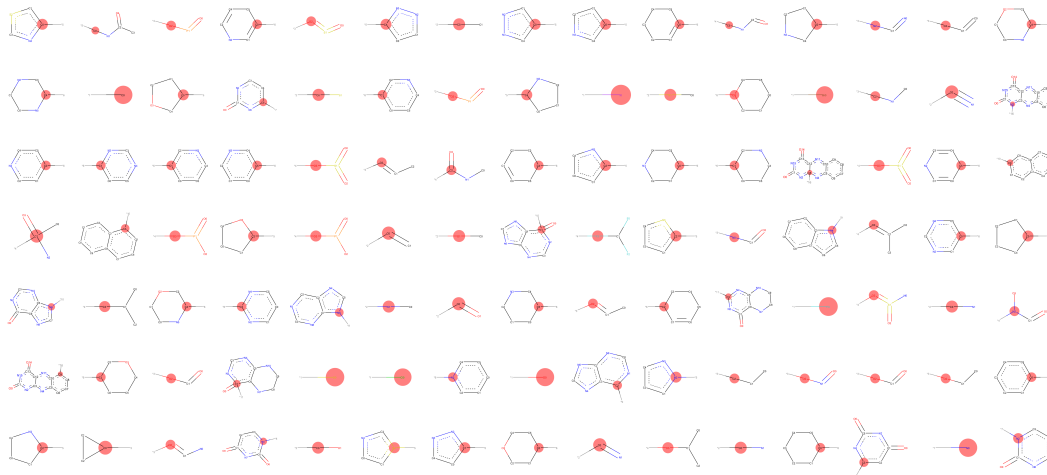


图 8: 分子设计中使用的构建块列表。茎，即连接块与分子其余部分的原子，被突出显示。

我们基于代理的预测来计算奖励。该代理是在一个包含 30 万随机生成分子的数据集上训练的，这些分子与目标蛋白的结合亲和力已通过 AutoDock (Trott 和 Olson, 2010) 计算得出。由于结合亲和力是一个能量值，越低越好，我们取其相反数，然后重新归一化（减去均值，除以标准差）以获得奖励。我们使用 sEH 蛋白及其 4JNC 抑制剂。可溶性环氧化物水解酶，或 sEH，是一种已被广泛研究的蛋白质，在呼吸系统和心脏疾病中起作用，使其成为一种有趣的药理学靶标和基准测试对象。需要注意的是，我们也尝试了其他生物相关的量，特别是 logP（正辛醇-水分配系数）和 QED (Bickerton 等, 2012)。两者都很容易通过 GFlowNet 最大化。对于 logP，我们很快找到了 logP 大于 20 的分子，这一点在生物学上已经不再有趣（作为参考，布洛芬的 logP 在 3.5 到 4 之间）。对于 QED，我们也迅速找到了具有最大可能 QED 值的分子，这个值在我们的动作空间中是 0.948（实际上，我们的前 1000 个分子在看到 10 万个分子后 QED 值大于 0.94）。由于对接是一个更难的预言者，我们专注于它。还需要注意的是，我们尝试了将不同分数乘法组合起来的方法（例如，将对接分数乘以重新归一化的 QED 和合成性），并取得了一定的成功。在这方面的一个更具体的贡献留待未来工作。

A.4 分子域实现细节

对于奖励定义的预言代理，我们使用一个 MPNN (Gilmer 等人, 2017)，其接收原子图为输入。原子图通过 RDKit 计算得到。图中的每个节点具有包括原子数的一热向量、杂化类型、隐含氢的数量以及作为受体或供体原子的二进制指示符在内的特征。MPNN 在每次迭代中使用 GRU 作为图卷积层，并迭代应用 12 步，随后进行 Set2Set 操作以减少图，最后是一个三层 MLP。其所有部分均使用 64 个隐藏单元，并在各处使用 LeakyReLU 激活函数（GRU 内部除外）。在非主动学习实验中，我们使用包含 300k 分子的数据集训练代理。为了使任务有趣，我们使用 80% 从随机轨迹获得的分子和 20% 从前次 RL 代理运行中获得的分子。这额外的 20% 包含了一些分数略高的分子。

足够多样化以构成一个有趣的挑战。请参见图 3 所示的奖励分布。确切的数据集已在我们的 github 仓库中提供，以便于复现。对于流动预测器 F ，我们也使用了一个 MPNN，但它接收的是块图作为输入。该图由构造而成的树。图中的每个节点都是一个学习到的嵌入（每个 105 个块都有自己的嵌入，每种键合类型都有边嵌入）。我们再次在卷积层上应用了 10 次 GRU。对于图中的每个分支（代表一个原子或代理可以连接新块的块），我们将对应的嵌入（10 步图卷积+GRU 的输出）传递给一个三层 MLP，以生成 105 个 logits，表示将每个块连接到此分支的概率（对于 MARS 和 PPO），或者表示流动 $F(s, a)$ （对于 GFlowNet）；由于每个块可以有多个分支，这个 MLP 还接收与该分支对应的块内的原子。对于停止动作，我们执行全局平均池化，然后通过一个三层 MLP 输出每个流动预测的 1 个 logit。我们在所有地方都使用了 256 个隐藏单元以及 LeakyReLU 激活函数。为了进一步提高稳定性，我们发现将终端转换的损失乘以一个因子 $\lambda_T > 1$ 有所帮助。直观地说，这样做优先考虑了流动终点处的正确预测，这些预测可以通过我们的引导目标在整个网络/状态空间中传播。这类似于在强化学习中使用奖励预测作为辅助任务（Jaderberg 等，2017）。以下是流动模型超参数的总结：

Learning rate	5×10^{-4}	# of trajectories per SGD step
Minibatch size	4	
Adam β, ϵ	$(0.9, 0.999), 10^{-8}$	
# hidden & # embed	256	
# convolution steps	10	ϵ in (12)
Loss ϵ	2.5×10^{-5}	
Reward T	8	
Reward β	10	
Random action probability	0.05	exploratory factor
λ_T	10	leaf loss coefficient
R_{min}	0.01	R is clipped below R_{min} , i.e. $\hat{R}_{min} = (R_{min}/T)^\beta$

对于 MARS，我们使用的学习率为 2.5×10^{-4} ，而对于 PPO，为 1×10^{-4} 。对于 PPO，我们使用的熵正则化系数为 10^{-6} ，并将奖励 β 设置为 4（更高值没有帮助）。对于 MARS，我们使用与 Xie 等（2021）相同的算法超参数。对于 JT-VAE，我们直接使用 Jin 等（2020）提供的代码，仅替换奖励信号。

附录 A.5 多轮实验

算法 1 定义了训练策略的程序 π_θ ，并在超网格和分子领域多轮实验的内循环中使用。在多轮设置下，多样化的生成效果变得明显。由于代理本身是基于生成模型提出的输入样本（并通过预言机评分，例如使用对接方法）进行训练的，如果生成模型探索性不足，奖励（由代理定义）只会围绕已发现的模式提供有用的信号。预言机的结果 $O(x)$ 被缩放为正值，并且

超参数 β (一种逆温度形式) 可用于使奖励函数的模式更加尖锐或平坦。

Algorithm 1: Multi-Round Active Learning

Input: Initial dataset $D_0 = \{x_i, y_i\}, i = 1, \dots, k$; K for *TopK* evaluation; number of rounds (outer loop iterations) N ; inverse temperature β

Result: A set $TopK(D_N)$ of high valued x

Initialization:

Proxy M ;

Generative policy π_θ ;

Oracle O ;

$i = 1$;

while $i \leq N$ **do**

 Fit M on dataset D_{i-1} ;

 Train π_θ with unnormalized probability function $r(x) = M(x)^\beta$ as target reward;

 Sample query batch $B = \{x_1, \dots, x_b\}$ with $x_i \sim \pi_\theta$;

 Evaluate batch B with $O, \hat{D}_i = \{(x_1, O(x_1)), \dots, (x_b, O(x_b))\}$;

 Update dataset $D_i = \hat{D}_i \cup D_{i-1}$;

$i = i + 1$;

end

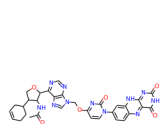
A.5.1 超网格

本文使用来自 `botorch`² 的高斯过程实现作为代理。每轮生成样本的查询批大小为 16。训练生成模型的超参数设置为单轮实验中表现最佳的值。初始数据集仅包含 4 种模式。GFlowNet 在 5 轮内发现了 10 种模式，而 MCMC 在 10 轮内发现了 10 种模式，相比之下，PPO 最终仅发现了 8 种模式 (含 $R_0 = 10^{-1}$)。

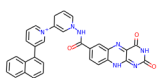
A.5.2 分子

初始分子集合 D_0 从 300k 数据集中随机选取 2000 个分子。在每一轮中，对于 MPNN 代理模型的重新训练，我们使用一个固定的验证集来确定停止条件。这个包含 3000 个样本的验证集同样是从 300k 数据集中随机选取的。我们在拟合生成模型时使用较少的迭代次数，其余超参数与单轮设置相同。

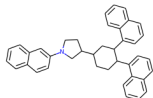
method	Reward after 1800 docking evaluations	
	top-10	top-100
GFlowNet	8.83 ± 0.15	7.76 ± 0.11
MARS	8.27 ± 0.20	7.08 ± 0.13



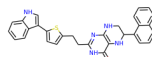
(a) Reward=8.26



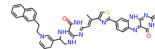
(b) Reward=9.18



(c) Reward=9.08



(d) Reward=8.73



(e) Reward=8.64

图 9: (a) 多轮分子实验中 D_0 获得最高奖励的分子。(b) 由 GFlowNet 生成的获得最高奖励的分子。(c)-(e) 来自 GFlowNet 生成的排名前十的分子样本。

²<http://botorch.org/>

A.6 超网格实验

首先，我们来看看 GFlowNet 学到了什么。所学到的流分布是什么？首先，在图 10（左）中，我们可以观察到所学到的分布 $\pi_\theta(x)$ 几乎完美地匹配在网格上 $p(x) \propto R(x)$ ，其中

$n = 2, H = 8$ 。在图 10（中）中，我们绘制了通往模式 $s = (6, 6)$ 的所有路径上的访问分布，起点为 $s_0 = (0, 0)$ 。可以看到，分布较为分散，但并非均匀：似乎有一些偏好其他角落，这可能是由于学习初期的偏差以及其它模式的位置所致。在图 10（右）中，我们绘制了从 $(0, 0)$ 到 $(6, 6)$ 的所有路径上的均匀分布作为参考。请注意，我们的损失函数并不强制任何类型的流分布，均匀流并不一定是理想的（调查这一点可能是未来工作的有趣方向，或许某些流分布具有更好的泛化特性）。

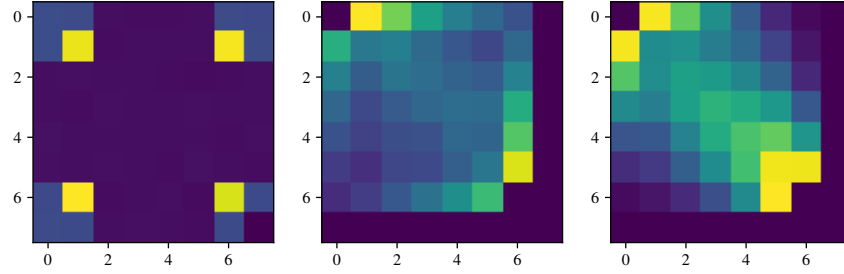


图 10: 包含 $n = 2, H = 8$ 的网格。左图，网格上学习到的分布 $\pi_\theta(x)$ 几乎完美匹配 $p(x)$ ；通过采样 30k 点进行测量。中图，导向 $(6, 6)$ 的采样路径上的访问分布。右图，导向 $(6, 6)$ 的所有路径上的均匀分布。

请注意，我们也在该领域运行了 Soft Actor Critic (Haarnoja 等人, 2018)，但我们未能找到超参数使 SAC 发现所有模式的 $n = 4, H = 8$ ；即使强烈正则化，SAC 最多也只能找到 16 个模式中的 10 个（但不至于使策略简单地变成均匀策略）。虽然我们认为我们的实现是正确的，但我们认为将这些结果包含在图中并不相关，因为它们表现不佳但也不令人惊讶：正如与奖励最大化一致，SAC 迅速找到了一个模式并集中其所有概率质量于该模式，这是我们试图通过 GFlowNet 避免的 RL 无多样性失败模式。

接下来，我们将损失作为函数 R_0 进行分析，在 $n = 4, H = 8$ 设置下再次观察。我们将损失分为两个部分：叶损失（终端状态的损失）和内部流损失（非终端状态的损失）。在图 11 中可以看到，随着 R_0 减小，内部流损失和叶损失都变大。这有两个原因：首先，例如，当 $R_0 = 10^{-2}$ ， $\log 10^{-2}$ 是一个较大的数值时，这对 DNN 准确输出来说更困难；其次，对于那些 $\log 10^{-3}$ 作为流输出的终端状态，在 $100\times$ 情况下比在 $R_0 = 10^{-1}$ 情况下更为罕见（因为我们是在策略上采样状态），因此 DNN 不太倾向于正确预测它们的价值。这导致了罕见但幅度较大的损失。需要注意的是，这些损失仍然很小，大约在 10^{-3} 或以下的量级，并且此时分布已经基本拟合，模型只是在收敛。

作为离线非策略方法的 GFlowNet 为了展示 GFlowNet 的这一特性，我们在固定的数据集轨迹上对其进行训练并观察所学分布。在这个实验中，我们使用 $R(x) = 0.01 + \prod_{i=1}^n (\cos(50x_i) + 1)f_N(5x_i)$ ， f_N 是正态概率密度函数， $n = 2$ 和 $H = 30$ 。我们展示了两个随机数据集的结果。首先，在图 12 中，我们展示了当数据集从均匀随机策略中采样时所学到的内容；其次，在图 13 中，当数据集通过在网格上均匀采点并向后走到根节点生成轨迹时所学到的内容。第一个设置应该比第二个设置困难得多，确实，当数据集中的点更加均匀时，所学分布与 $p(x)$ 匹配得更好。请注意，在这两种情况下，许多点被有意地排除在外，作为泛化测试。

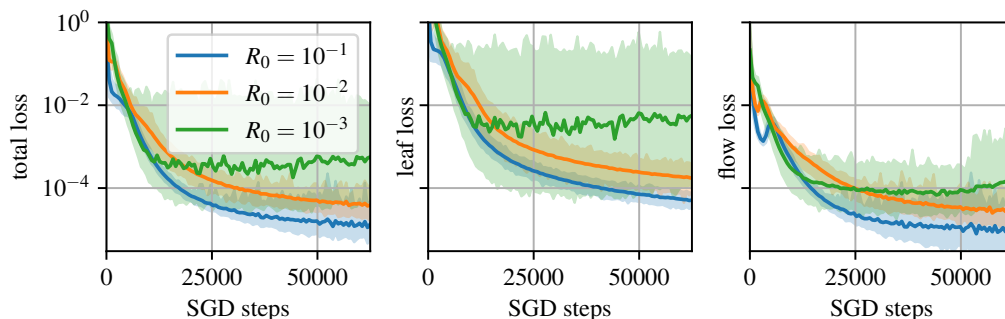


图 11: 在超网格中“角落”奖励函数训练期间的损失，其中 $n = 4$, $H = 8$ 。阴影区域是最小最大边界。

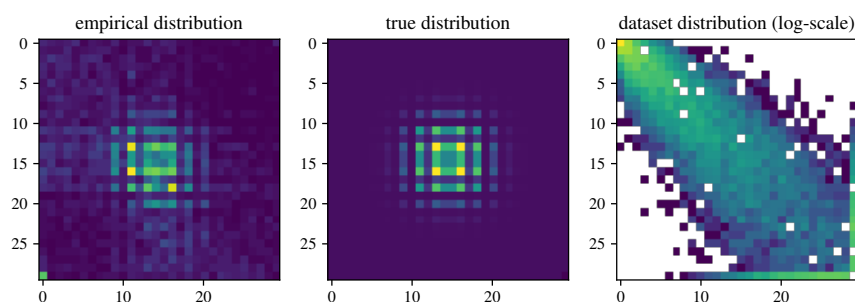


图 12: 网格上的 $n = 2$, $H = 30$ 。左图，所学分布 $\pi_\theta(x)$ 。中图，真实分布。右图，数据集分布，这里通过执行从 s_0 开始的均匀随机策略生成。

这些结果表明，GFlowNet 可以轻松应用于离线和非策略场景。请注意，我们在这两个图上没有进行超参数搜索，这些图仅用于说明，我们认为通过调整超参数可能会实现更好的泛化。

附录 A.7 GFlowNet 在分子域的结果

在这里，我们提供额外的结果，以揭示我们的方法 GFlowNet 所学到的内容。让我们首先查看图 4 中的数值结果：

method	Reward at 10^5 samples		
	top-10	top-100	top-1000
GFlowNet	8.36 ± 0.01	8.21 ± 0.03	7.98 ± 0.04
MARS	8.05 ± 0.12	7.71 ± 0.09	7.13 ± 0.19
PPO	8.06 ± 0.26	7.87 ± 0.29	7.52 ± 0.26
	Reward at 10^6 samples		
	top-10	top-100	top-1000
GFlowNet	8.45 ± 0.03	8.34 ± 0.02	8.17 ± 0.02
MARS	8.31 ± 0.03	8.03 ± 0.08	7.64 ± 0.16
PPO	8.25 ± 0.12	8.08 ± 0.12	7.82 ± 0.16
Reward for 10^6 -equivalent compute			
JT-VAE + BO	6.03	5.86	5.31

这些是经过 3 次运行计算出的均值和标准差。可以看出 GFlowNet 生成的分子显著更好。它还生成了更多样化的分子：GFlowNet 的均值

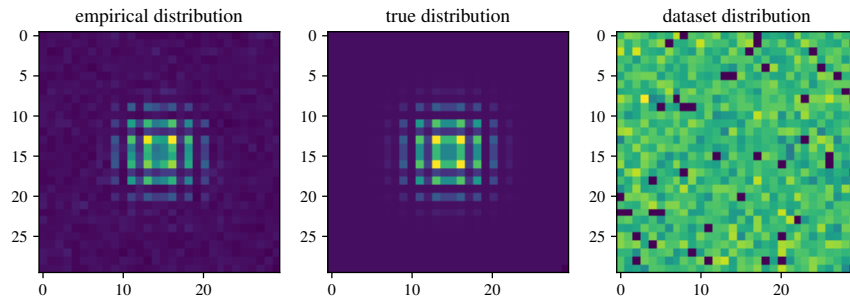


图 13: 包含 $n = 2$, $H = 30$ 的网格。左图, 学习到的分布 $\pi_\theta(x)$ 。中图, 真实的分布。右图, 数据集分布, 这里通过在网格上均匀采样一个点并随机选择父节点直到 s_0 来生成训练轨迹。

其排名前 1000 的分子与 0.44 ± 0.01 、PPO、 0.62 ± 0.03 以及 MARS、 0.59 ± 0.02 之间的成对 Tanimoto 相似性 (3 次运行的均值和标准差)。在这个环境中, 随机代理将产生平均成对相似性为 0.231 的结果 (并且奖励非常低)。

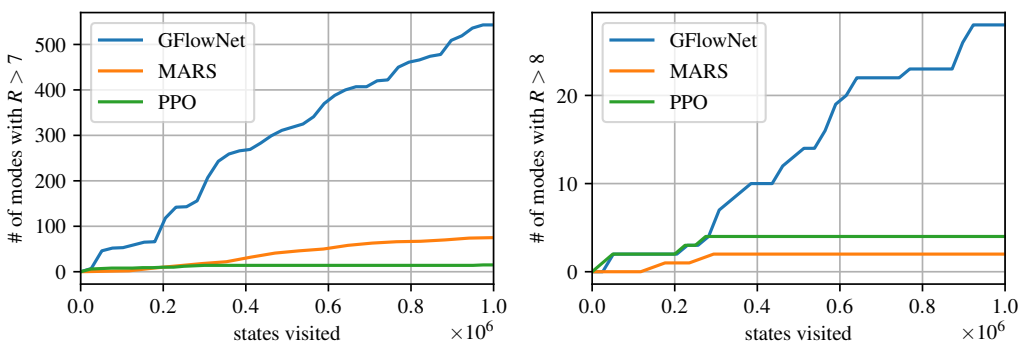


图 14: 找到的高于奖励阈值 T 的 Tanimoto 分离模式数量作为已见分子数量的函数。参见正文。左图, $T = 7$ 。右图, $T = 8$ 。

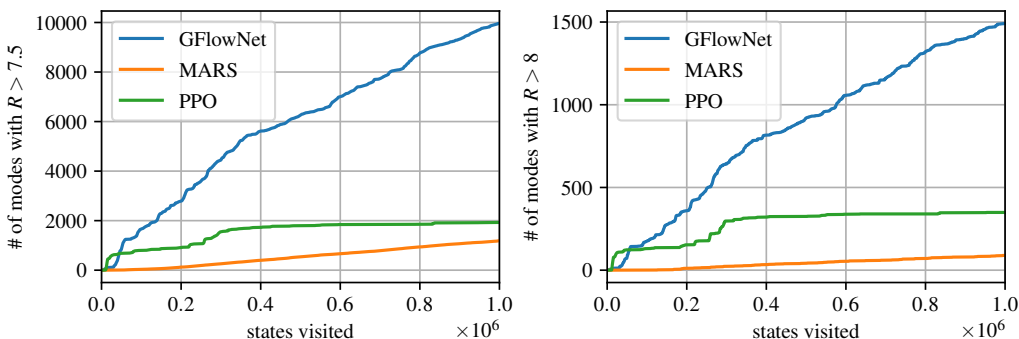


图 15: 找到的高于奖励阈值 T 的多样 Bemis-Murcko 支架 (Bemis 和 Murcko, 1196) 数量作为已见分子数量的函数。左图, $T = 7.5$ 。右图, $T = 8$ 。

我们还发现 GFlowNet 生成了更多样化的分子, 通过近似计数高奖励分子内的模式数量。在这里, 我们将“模式”定义为能量高于某个阈值 T 的分子, 在 Tanimoto 空间中彼此最相似度不超过阈值。换句话说, 当我们发现一个新的分子, 其与之前找到的所有模式代表分子的

Tanimoto 相似性小于 S 时，我们认为找到了一个新的模式代表分子。我们选择 Tanimoto 相似性 S 的阈值为 0.7，因为这在药物化学中常用于寻找相似分子，并设置奖励阈值为 7 或 8。我们在图 14 中绘制了结果。可以看到对于 $R > 7$ ，GFlowNet 发现的模式数量远多于 MARS 或 PPO，超过 500 个，而 MARS 仅发现了不到 100 个。

另一种近似模式数量的方法是计算高于特定奖励阈值的分子中所含的不同 Bemis-Murcko 支架的数量。我们在图 5 中绘制了这些计数，再次看到 GFlowNet 发现的模式数量更多。

接下来，让我们尝试理解 GFlowNet 学到了什么。在一个大规模且无法访问 $p(x)$ 的领域中，证明 $\pi_\theta(x)$ 与期望分布 $p(x) \propto R(x)$ 相匹配并非易事。这是由于多路径问题：为了计算真实的 $p_\theta(x)$ ，我们需要对所有通往 x 的轨迹的 $p_\theta(\tau)$ 进行求和，而这些轨迹的数量可能极其庞大。相反，我们展示了各种测度，表明所学分布与假设的 $\pi_\theta(x)$ 与 $p(x) \propto R(x)^\beta$ 一致。

在图 16 中展示了如何 F_θ 部分地学会匹配 $R(x)$ 。特别地，我们绘制了叶子的流入量（即对于叶子 $s, a: T(s, a) = s'$ $F(s, a)$ s' ，其 $\sum$ ）作为对比目标分数 $(R(x)^\beta)$ 。

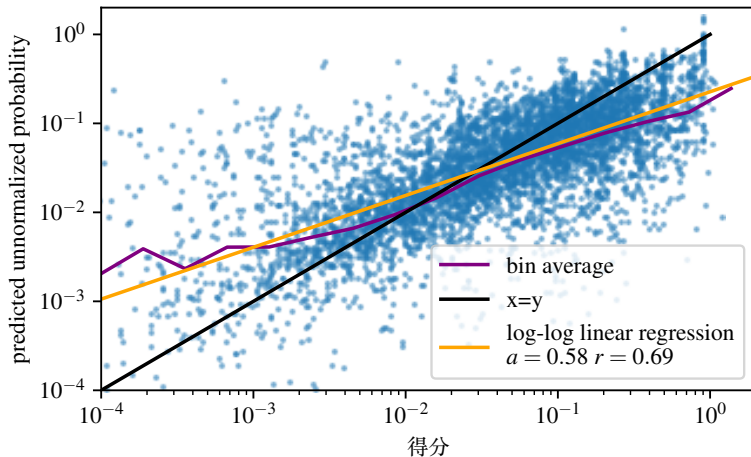


图 16: 分数 $(R(x)^\beta)$ 与叶片流入量（预测的未归一化概率）的散点图。两者应匹配。我们看到对数-对数线性回归的斜率为 0.58，且 r 为 0.69。斜率小于 1 表明 GFlowNet 倾向于低估高奖励（这是合理的，因为高奖励由于其稀有性而较少被访问），但仍然合理地拟合其数据。这里 $\beta = 10$ 。此处绘制的是一个运行生成的最后 5000 个分子。

另一种看待学习到的概率具有自洽性的方法是直方图中的 $R(x)/Z$ 和 $\hat{p}_\theta(x)/Z$ 相匹配，其中我们使用预测的 $Z = \sum_{a \in \mathcal{A}(s_0)} F(s_0, a)$ ，而 $\hat{p}_\theta(x)$ 是叶节点 x 的流入量，如同上述。我们在图 17 中展示了这一点。

从损失角度来看，有趣的是我们的模型在深度强化学习的价值预测方面表现出类似的行为，即价值损失永远不会降至 0。这在一定程度上是由于引导和状态空间的大小所致。确实，在我们的超网格实验中，随着模型收敛，损失会降至 0。我们在图 18 中分别绘制了叶转移（其中流入被训练以匹配奖励）和内部流转移（在访问过的状态下，流入被训练以匹配流出）的损失。

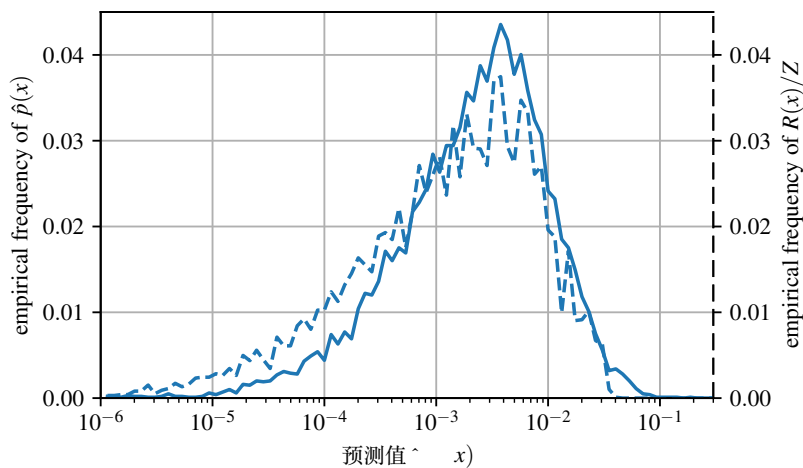


图 17: 预测密度的直方图与奖励直方图。两者应匹配。这些计算基于运行生成的最后 10000 个分子。此图再次表明模型欠拟合。它认为低奖励分子比实际更不可能，或者相反，认为低奖励分子比实际更好。这与之前显示斜率低于 1 的图一致。

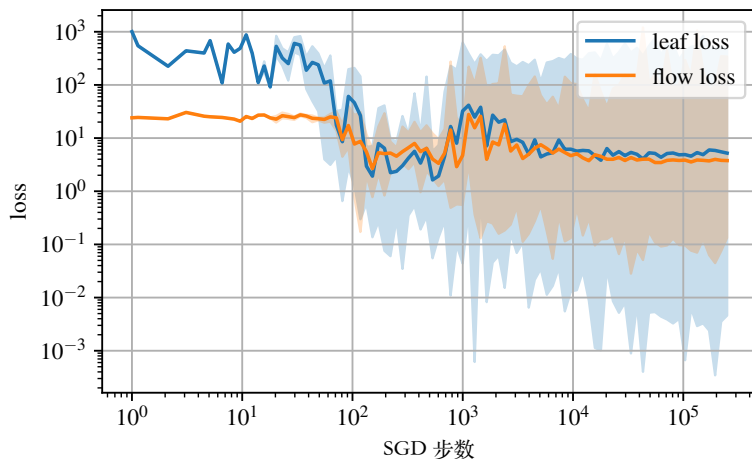


图 18: GFlowNet 在分子域上典型运行的损失随训练的变化。阴影区域表示区间内的最小值和最大值。我们注意到几个阶段：初始阶段，预测的比例不正确，叶损失非常高。随着预测比例调整，我们观察到第二阶段，流动变得一致，并观察到损失下降。然后，当模型开始发现更多有趣的样本时，损失上升，随后下降，因为它开始正确地适应各种样本上的流动。由于状态空间巨大，缺乏收敛是预期的；这类似于深度强化学习中 Atari 等领域的基于价值的方法。